Palacan

WHITEPAPER

Security Whitepaper

The complete technical description of Palacan: the protocol, the architecture, what the server can see, the threat model and how to verify everything.

This document is generated from the public repository, where the most recent version is always available: github.com/Sbirringa/palacan-client

Version of 9 November 2026, from commit `d1261f8`

Source: `docs/PROTOCOL.md`, `docs/ARCHITECTURE.md`, `docs/VERIFICATION.md`

© 2026 Palacan. All rights reserved.

Contents

01 Palacan protocol

- Building blocks
- Keys
- Hybrid signatures
- Post-quantum prekeys
- Room keys and messages
- Sealed delivery of room keys (hiding the sender)
- Calls
- Verifying contacts
- Key backup
- Storage on the device
- Formal verification
- Known limits

02 Palacan architecture

- Components
- What runs where
- What the server can see
- Threat model
- Formal and automated checks

03 Verifying Palacan

- The website is exactly this code
- Formal proofs
- Cryptography module
- Web app tests
- What you cannot verify from here

Palacan protocol

This document describes how Palacan encrypts messages and manages keys. It is written for security researchers and for anyone who wants to check the code. Everything described here runs on the user's device, in the code of this repository. The server only stores and forwards data that is already encrypted.

The formal proofs of the main properties, and the limits we know about, are in `formale/README.md`. The overall design and what the server can see are in `ARCHITECTURE.md`.

Note: identifiers in the code are mostly in Italian. Where it helps, this document gives the Italian name in brackets.

---

1. Building blocks

Purpose	Primitive	Implementation
One-to-one sessions between devices	Olm (triple Diffie-Hellman handshake + Double Ratchet, Curve25519, Ed25519)	vodozemac (Rust, audited), compiled to WebAssembly in `crypto/palacan-crypto`
Group and chat messages	Megolm (symmetric ratchet, AES-256 + HMAC-SHA-256, Ed25519-signed messages)	vodozemac
Post-quantum key encapsulation	ML-KEM-768 (FIPS 203)	`libcrux-ml-kem` (Cryspen, formally verified with hax/F*)
Post-quantum signatures	ML-DSA-65 (FIPS 204), hedged	`libcrux-ml-dsa` (Cryspen)
Post-quantum envelope	HKDF-SHA-256 + XChaCha20-Poly1305	`crypto/palacan-crypto/src/pq.rs`
Sealed boxes for metadata	X25519 + HKDF-SHA-256 + AES-256-GCM	Web Crypto API (non-extractable keys)
Local storage	AES-256-GCM with a non-extractable device key	Web Crypto API + IndexedDB
Key backup	Argon2id + AES-256-GCM	in the browser, before upload

The wire formats of Olm and Megolm are those of libolm version 1, so the code stays compatible with the format used before the move to vodozemac. Curve25519 keys of low order, which would make a Diffie-Hellman result predictable, are rejected before a session

is opened (`crypto/palacan-crypto/tests/chiavi_deboli.rs`).

---

2. Keys

2.1 Device keys

Each device (browser) has:

- an **Olm account**: a Curve25519 identity key, an Ed25519 signing key and one-time keys;
- an **ML-DSA-65 signing key** (32-byte seed). Its public key is published together with a *binding proof*: a hybrid signature (section 3) of `palacan-pqfirma-legame-v1|<user>|<curve25519 key>`, so the Ed25519 and ML-DSA keys vouch for each other. Inside Olm messages only the SHA-256 fingerprint of the ML-DSA key travels; whoever learns it downloads the key by fingerprint and accepts it only if the fingerprint matches and the binding proof verifies (`src/utils/firmaPq.js`);
- **ML-KEM-768 one-time prekeys** and one **reserve (fallback) prekey** (section 4);
- an **envelope ("busta") key pair**: X25519 (non-extractable) and ML-KEM-768, used for sealed boxes (section 6);
- a **device storage key**: AES-256-GCM, non-extractable, kept in IndexedDB (section 10).

The Ed25519 key of another device is learned **inside** Olm messages (an author attestation in the encrypted part), never only from the server.

2.2 Principal key (cross-signing)

Each person has one **principal key** (`chiave principale`): an Ed25519 key (in a dedicated Olm account that is used only for signing) plus an ML-DSA-65 key (`src/utils/chiavePrincipale.js`).

- It is published with a hybrid **self-signature** of `palacan-principale-v1|<user>|<ed25519>|<ML-DSA fingerprint>`.
- It signs each of the person's devices: `palacan-firma-dispositivo-v1|<user>|<device id>|<curve25519 key>`.
- Senders trust the person once (safety number, section 8) and then trust every device signed by that principal key. A device that is **not signed** is shown as such and **does not receive room keys**.

- A new device gets the principal key from the encrypted backup (section 9) or is approved from an existing device by comparing a code.
- The principal key never replaces itself silently. If it changes, contacts see an identity-change warning.

---

3. Hybrid signatures

Every signature that matters for trust is **hybrid**: valid only if **both** Ed25519 **and** ML-DSA-65 verify (`crypto/palacan-crypto/src/firma_pq.rs`). Both algorithms sign the same text:

```
palacan-firma-ibrida-v1|<context>|<Ed25519 public key>|<SHA-256 hex of the ML-DSA-65 public key>|<message>
```

- Binding both public keys into the text prevents stripping one signature or mixing keys.
- The context (only `[a-z0-9-.:_]`, at most 64 characters) separates the different uses. ML-DSA-65 additionally uses `palacan-firma-ibrida-v1` as its FIPS 204 context string.
- ML-DSA signatures are hedged (fresh randomness for each signature).
- The format is proven unambiguous with Kani (`src/formato_pq.rs`): two signatures over different contexts, keys or messages never sign the same text.

---

4. Post-quantum prekeys

Each device publishes ML-KEM-768 public keys (`src/utils/chiaviPq.js`):

- **one-time prekeys**, published in batches of 50, at most 100 kept by the server per device; the server hands each one out **once** and deletes it;
- one **reserve prekey**, handed out only when the one-time prekeys have run out. It is replaced every 7 days.

Each prekey is signed by the device, with Ed25519 and with the hybrid signature, over:

```
palacan-pq-prechiave-v2|<user>|<device id>|<key id>|<type>|<public key>|<creation time in ms>
```

Renewal. Unused one-time prekeys are replaced every 7 days: the device publishes a new batch and the server deletes the old unused ones. The private seeds of retired prekeys are kept for 30 days, for envelopes already on the way, and then deleted. The seed of a

prekey that has been used is deleted as soon as the envelope is opened.

Age check. A sender refuses prekeys older than **21 days** or dated more than one day in the future. A server therefore cannot keep serving an old, possibly stolen prekey for ever. Prekeys signed without a date (format `v1`, before this rule) are accepted until 10 December 2026 and refused afterwards. Every device republishes its prekeys with a date the first time it opens after the update.

---

5. Room keys and messages

Every chat (one-to-one or group) uses **Megolm**:

- each sender has an outbound Megolm session per chat; every message is encrypted and **signed** with the session key, so a member who only knows the ratchet key cannot write in someone else's name;
- the session is replaced after **100 messages** or **7 days**, and whenever a device leaves the chat or membership changes. A device that has left does not receive the new session.

The Megolm session key (the **room key**) is sent to **each signed device** of each member over **Olm**, inside a mandatory **post-quantum envelope** (`src/utils/bustePq.js`):

1. the sender claims one ML-KEM prekey per recipient device and checks its signatures and age (section 4);
2. the room-key message (`m.room_key`, and also `m.forwarded_room_key(s)` for forwarded keys) is sealed with ML-KEM-768 and the result is what Olm encrypts. The envelope is:
3. a fresh ML-KEM-768 encapsulation for every envelope;
4. key = HKDF-SHA-256(shared secret, salt = SHA-256(domain ‖ public key ‖ ciphertext), info = domain ‖ context), with domain `palacan-pq-busta-v1`;
5. XChaCha20-Poly1305 with a random 24-byte nonce, and the context as associated data;
6. format: version (1 byte) ‖ ML-KEM ciphertext (1088 bytes) ‖ nonce (24 bytes) ‖ ciphertext + tag;
7. context: `palacan-pq-chiave-v1|<sender>|<sender curve25519>|<recipient>|<recipient curve25519>`, so an envelope cannot be moved to another conversation or device.

To read a room key, an attacker must break **both** X25519 (Olm) **and** ML-KEM-768. This protects against "harvest now, decrypt later": traffic recorded today stays protected even against a future quantum computer.

No downgrade. A room key is never sent without the post-quantum envelope, not even at first contact, and a room key that arrives without one is refused. A device that has not published prekeys yet receives the key at the next share, once it publishes them. The chat messages themselves use only symmetric cryptography (Megolm), which quantum computers do not break in practice.

Missing keys. A device that missed a room key (for example, because it was offline) asks the members of the chat for it. Forwarded keys travel in the same post-quantum envelope.

---

6. Sealed delivery of room keys (hiding the sender)

To keep the server from seeing **who** sends a room key to whom, the Olm message is put in a **sealed box** (`busta sigillata`) addressed to the recipient's envelope key (`src/utils/busta.js`, `src/utils/busteSigillate.js`). Sender, sender identity key and Olm content are **inside**: the server only sees the recipient.

- **v1** (X25519): ephemeral X25519 key; key = HKDF-SHA-256(shared secret, salt = ephemeral public ‖ recipient public, info `palacan-busta-v1`); AES-256-GCM with associated data `palacan-busta-v1`. Format: `0x01 ‖ ephemeral public (32) ‖ nonce (12) ‖ ciphertext + tag`.
- **v2** (hybrid, used when the recipient has an ML-KEM envelope key): the v1 box sealed again with the post-quantum envelope of section 5, context `palacan-busta-v2|<recipient X25519 key>`, prefixed with `2:`.
- The recipient checks that the identity key inside the box is the one published for the claimed sender.
- **Signed envelope keys.** Each person's envelope keys are signed by their principal key (context `busta`, text `palacan-busta-v1|<user>|<X25519 key>|<ML-KEM key>`). If the sender has already seen the recipient's principal key, or the server delivers a valid one, the box is used **only** with a valid signature. Otherwise the room key goes the usual way (Olm to signed devices only), without hiding the sender.
- **Group invitations.** The name of a private group in an invitation is sealed in the same way, so the server never sees it.

---

7. Calls

Voice and video calls use WebRTC. The call signalling (offer, answer, ICE candidates) is encrypted end to end with the chat's Megolm key and authenticated, with replay protection and a validity of 5 minutes. The DTLS fingerprints travel encrypted, so the

server cannot put itself in the middle of a call. Media is always relayed through a TURN server, so the other person never sees the user's IP address.

---

8. Verifying contacts

- Device keys are trusted on first use (TOFU), per device. If a contact's key changes, the app shows a warning.
- The **safety number** and the **QR code** (version 2) are computed from the two people's principal keys. Comparing them face to face or on a call proves that nobody is in the middle. The QR check works in both directions.
- Towards a contact the user has verified, messages and calls are **blocked** if a key appears that was never seen before.

---

9. Key backup

The Olm account, the Megolm room keys, the ML-DSA seed and the principal key can be saved in an encrypted backup on the server:

- the backup key is derived from the user's passphrase with **Argon2id**; the backup is encrypted with AES-256-GCM in the browser before it is uploaded. The server never sees the passphrase or the keys;
- devices of the same account merge their backups instead of overwriting each other, so no room key is lost.

A new device can restore everything from the backup. Without a backup, it must be approved from an existing device (section 2.2).

---

10. Storage on the device

- On first start the browser creates a 256-bit AES-GCM **device key** that is **non-extractable**, so not even the app's own code can read it, and keeps it in IndexedDB.
- The Olm and Megolm state, the ML-KEM and ML-DSA seeds, the principal key, login data, drafts and the local message history are stored **encrypted** with that key, each item bound to its own name.
- The local history is deleted when the user logs out.

- In browsers without IndexedDB the app runs in a reduced mode ("reserve" mode) with weaker local protection.

This protects against someone reading the browser's files and against other websites. It does not protect against malware that controls the computer while the user is logged in.

---

11. Formal verification

- **ProVerif** (symbolic model): the post-quantum envelope against "harvest now, decrypt later", the hybrid signature, the Megolm and Olm ratchets, and the renewal of post-quantum prekeys after theft of a device's state.
- **Kani** (model checking of the Rust code that ships): the formats of the envelope and of the hybrid signature.
- **proptest** and **mutation testing** on the cryptography module.

Results, exact claims and limits: `formale/README.md`. How to rerun everything: `VERIFICATION.md`.

---

12. Known limits

- **Trust on first use.** Keys are learned the first time they are seen, as in Signal and Matrix. An attacker who already has a quantum computer when the keys are first learned is not covered, as with every hybrid protocol today.
- **Stolen device.** A stolen recipient device can read the Megolm history it had already received. After the theft of a device's state, Olm becomes secure again only after a round of new keys that the attacker does not tamper with. Post-quantum prekeys stolen with the state stay usable until they are renewed (at most 7 days with the app open), and an attacker who also controls the server can keep serving them for at most 21 days from their creation, or publish new ones signed with the stolen signing keys until the device is removed.
- **Signed envelope keys.** If the sender has never seen the recipient's principal key and the server does not deliver it, the envelope key is used without a signature (trust on first use).
- **Metadata.** See `ARCHITECTURE.md`, section "What the server can see".

Palacan architecture

This document explains where each part of Palacan runs, what each part can see, and which attacks Palacan is designed to resist. The cryptographic details are in `PROTOCOL.md`; how to check all of this yourself is in `VERIFICATION.md`.

1. Components

Component	Where it runs	In this repository
Web app (React, Vite)	in the user's browser	yes: `frontend/my-react-app/vite-project/src`
Cryptography module (Rust: vodozemac, ML-KEM-768, ML-DSA-65)	in the browser, as WebAssembly	yes: `crypto/palacan-crypto`
Static website hosting (nginx, strict Content-Security-Policy)	server	configuration only: `frontend/my-react-app/vite-project/nginx`, `frontend/my-react-app/vite-project/sicurezza`
Backend API and WebSocket relay (Python, FastAPI, PostgreSQL)	server	no
TURN relay for calls	server	no
Palacan Verifica browser extension	in the user's browser	yes: `frontend/my-react-app/vite-project/estensione`

Why the server code is not published. All encryption and decryption happen on the user's device, in the code of this repository, before anything leaves it. The server only stores and forwards data that is already encrypted and never has the keys. To check that messages are protected, what matters is the code running on the device, and that code is here, together with a way to prove that the website serves exactly this code (`VERIFICATION.md`).

2. What runs where

On the device:

- generating and storing all private keys (Olm, Megolm, ML-KEM, ML-DSA, principal key, envelope keys);

- encrypting and decrypting messages, reactions, call signalling, group names and room keys;
- verifying every signature (device keys, prekeys, principal keys, envelope keys, release signature of the website);
- the encrypted local history and the encrypted key backup.

On the server:

- accounts (username and password hash; no email or phone number is required), sessions and rate limits;
- public keys, signatures and the encrypted backup, stored and handed out without being able to forge them;
- relaying encrypted messages and keys between devices, and storing encrypted messages until they are delivered or expire;
- for companies: separate workspaces, admin roles, employee accounts and tasks.

---

3. What the server can see

The server **cannot** read messages, reactions, call content, private group names or room keys, and it cannot forge signatures. Some metadata is unavoidable for delivering messages. Palacan reduces it as follows:

- **Senders.** For private chats the server does not store who sent a message. Room keys are delivered in sealed boxes: the server sees the recipient, not the sender (`PROTOCOL.md`, section 6). Inside a company workspace the sender is not stored either, and the admin cannot look it up.
- **Times.** For private chats, stored times are rounded to the hour.
- **Length.** For private chats, messages are padded, so their exact length is hidden.
- **Group names.** Private group names are end-to-end encrypted, also in invitations.
- **Retention.** Delivery data, expired items and dead sessions are cleaned up automatically. Messages can delete themselves after a time the user chooses. Company messages are never deleted automatically.

What the server **can** still see:

- which accounts are members of which chats, and when devices are connected (transport-level IP addresses and timing of connections), while it is delivering messages;

- that a call is taking place between members of a chat, and its start and end. The TURN relay sees the IP addresses of the two sides (each side's address is hidden from the other, not from the relay);
- public keys and signatures, the number of prekeys left, and the size and time of backup uploads.

---

4. Threat model

Palacan is designed to protect against:

- a **network attacker** who records or modifies traffic (TLS, plus end-to-end encryption);
- a **malicious or compromised server**, including one that modifies the stored data: it cannot read content, cannot replace keys without the change being detected (signatures, principal key, safety numbers), cannot remove the post-quantum protection of room keys (no downgrade), and cannot serve modified website code to users of the Palacan Verifica extension without being detected;
- **"harvest now, decrypt later"**: an attacker who records traffic today and has a quantum computer in the future (hybrid X25519 + ML-KEM-768 for room keys and sealed boxes, hybrid Ed25519 + ML-DSA-65 signatures);
- the **theft of a device's state**, with limits: past messages stay protected (forward secrecy of the ratchets), and security comes back after key rotation (Megolm sessions, Olm ratchet, post-quantum prekey renewal);
- an attacker who reads the **browser's files** (non-extractable device key, encrypted local storage).

Out of scope or limited (also listed in `formale/README.md`):

- **Trust on first use.** Keys are learned the first time they are seen. Users who want certainty compare the safety number or scan the QR code.
- **The website is delivered by the server.** Users who do not use the Palacan Verifica extension cannot check the code before it runs; a change targeted at one person can only be detected afterwards. An installable app signed with the release key is planned.
- **The release key** is held by one person, encrypted with a passphrase. Whoever obtained both the key file and the passphrase could sign a release.
- **Malware on an unlocked device**, physical access to an unlocked device, and attacks on the user's operating system or browser.

- **Denial of service.** A malicious server can refuse to deliver messages, keys or calls; it cannot read or forge them.
- **Metadata** listed in section 3.

---

5. Formal and automated checks

Every change to the code runs automatically:

- unit tests of the web app (Vitest) and of the server (pytest), and end-to-end tests with real browsers (Playwright);
- the reproducible build and the publication of its fingerprints;
- ProVerif, Kani, proptest and mutation tests on the cryptography.

The full register of guarantees, with the commit and the proof for each one, is in `formale/README.md`. That register is written in Italian; this document and `PROTOCOL.md` summarise it in English.

Verifying Palacan

You do not have to trust us. This document explains how to check, yourself, that the Palacan website runs exactly the code of this repository, and how to rerun the proofs and tests of the cryptography.

---

1. The website is exactly this code

The build of the web app is **reproducible**: the same commit always produces the same files, byte for byte. The build writes the SHA-256 fingerprint of every file in `dist/palacan-impronte.json`, and that list is **signed** with the Palacan release key, a hybrid Ed25519 + ML-DSA-65 key.

With the browser extension (every time you open the site)

Install **Palacan Verifica** from `frontend/my-react-app/vite-project/estensione/` (instructions in `LEGGIMI.txt` in that folder; Chrome, Edge and Brave). Each time you open Palacan it checks:

- the signature of the fingerprint list, against the release key built into the extension;
- the fingerprint of the page and of **every** file the page loads, also later;
- the service worker, the Content-Security-Policy and the release number, which must never go backwards.

The icon is green if everything matches, red with the reason if something does not, and grey if it cannot check.

By hand (on your computer)

You need Node.js 22 and git.

```
git clone https://github.com/Sbirringa/palacan-client.git
cd palacan-client/frontend/my-react-app/vite-project
npm ci
npm run build
node scripts/verificaSito.mjs --confronta dist/palacan-impronte.json
```

`verificaSito.mjs` downloads the fingerprint list from the website, compares it with the build you have just made, and checks its signature against the public release key in `sicurezza/chiave-rilascio-pubblica.json`. Every file of the site must match.

On GitHub

Every change to this repository runs the workflow **"Build verificabile"**, which repeats the same build and publishes the fingerprint of `palacan-impronte.json` in the run summary (artifact `impronte-<commit>`, kept 90 days). That fingerprint must be the same as the one of the site.

The release key's fingerprints are: ML-DSA-65 `b17eb99909ede552…`, Ed25519 `laOmhJxypCT8…` (full key in `frontend/my-react-app/vite-project/sicurezza/chiave-rilascio-pubblica.json`).

---

2. Formal proofs

The proofs are in `formale/` and run on GitHub at every change (workflow **"Modelli formali"**). To rerun them yourself:

ProVerif (protocol models; needs opam):

```
opam install proverif
python3 formale/proverif/verifica.py
```

`verifica.py` checks that every query gives exactly the expected result: a property that is no longer proven, or an attack that the model no longer finds, makes it fail.

Property tests and mutants (need Rust and Python):

```
cd crypto/palacan-crypto
cargo test --test proprieta_pq
cd ../..
python3 formale/mutanti/prova_mutanti.py
```

Kani (Linux or macOS):

```
cargo install --locked kani-verifier
cargo kani setup
cd formale/kani
cargo kani
```

3. Cryptography module

```
cd crypto/palacan-crypto
cargo test
```

The tests include the official test vectors of ML-KEM-768 (FIPS 203) and ML-DSA-65 (FIPS 204) from independent implementations, compatibility with libolm, and the rejection of weak Curve25519 keys. The workflow **"Modulo crittografico"** runs them at every change.

4. Web app tests

```
cd frontend/my-react-app/vite-project
npm ci
npx vitest run
```

Many tests use the real cryptography (the compiled WebAssembly module), not mocks.

5. What you cannot verify from here

The server code is not public. This does not weaken the end-to-end encryption, which you can verify here, but you cannot check what the server does with the metadata it sees (`ARCHITECTURE.md`, section 3).